

Fonctions asynchrones et API Fetch

Principes de base des fonctions asynchrones

Les fonctions asynchrones sont des fonctions qui ne **bloquent pas l'exécution du programme** lorsqu'elles sont appelées. Elles sont utilisées pour effectuer des opérations qui prennent du temps, comme les appels au serveur, les accès à la base de données, etc. Lorsque une fonction asynchrone est appelée, le programme continue à s'exécuter pendant que la fonction asynchrone effectue son travail en arrière-plan. Une fois que la fonction asynchrone a terminé son travail, elle appelle une **fonction de rappel (callback)** pour traiter les résultats.

Dans la session précédente, nous avons vu la fonction **setTimeout** qui est une fonction asynchrone car elle ne bloque pas l'exécution du programme et permet de définir un délai avant d'exécuter une fonction.

Si nous n'utilisons pas de fonctions asynchrones pour les opérations qui prennent du temps, notre programme risque de se bloquer et de ne pas répondre aux interactions de l'utilisateur. **alert** est un exemple de fonction synchrone qui bloque l'exécution du programme jusqu'à ce que l'utilisateur ferme la boîte de dialogue. Cela implique que l'interface utilisateur ne sera pas réactive et que l'expérience utilisateur sera mauvaise, les éléments ne s'afficheraient pas correctement, les animations ne se dérouleraient pas correctement, etc.

```
alert("Ceci est une alerte"); //Bloque l'exécution du programme jusqu'à ce que
l'utilisateur ferme la boîte de dialogue
```

Fonction de rappel (Callback)

Les fonctions de rappel sont des fonctions qui sont passées en tant que paramètre à une autre fonction et qui sont exécutées lorsque la fonction principale a terminé son exécution. Cela permet de changer le comportement de la fonction principale en fonction des besoins de l'application. On appelle la fonction principale et on passe une fonction de rappel différente en fonction des besoins.

Dans l'exemple suivant, la fonction **fctRappel** contient une opération asynchrone (setTimeout) qui appelle la fonction de rappel **apresAsynchrone** lorsque les opérations sont terminées. Les opérations asynchrones utilisent généralement des fonctions de rappel pour traiter les résultats des opérations.

```
//Exemple de fonction de rappel
function operationAsynchrone(fctRappel) {
  //Effectuer des opérations asynchrones
  setTimeout(function () {
    //Appeler la fonction de rappel
    fctRappel();
  }, 1000);
}

function apresAsynchrone() {
  console.log("Après l'opération asynchrone");
}
```

```
//Appeler la fonction de rappel  
operationAsynchrone(apresAsynchrone);
```

Autres exemples de fonctions asynchrones

Les fonctions asynchrones sont également utilisées pour effectuer des opérations qui prennent du temps, comme l'accès à la base de données, la lecture de fichiers, etc. **À noter que certaines API Web ne sont pas disponibles dans tous les navigateurs.**

- [Battery Status API](#)
- [Documentation FileReader](#)
- [File System Access API](#)
- [Geolocation API](#)
- [WebSockets](#)
- [Press-papier](#)
- [Pour une liste exhaustive des API Web](#)

```
//Exemple de fonction asynchrone avec Battery Status API, pour obtenir le niveau  
de batterie. Le résultat est obtenu de manière asynchrone via une promesse.  
navigator.getBattery().then(afficherNiveauBatterie);  
  
function afficherNiveauBatterie(battery) {  
    console.log("Niveau de batterie : " + battery.level * 100 + "%");  
}  
  
navigator.geolocation.getCurrentPosition(gestionLocalisation);  
//Fonction de rappel pour gérer la localisation, est appelée lorsque la  
localisation est obtenue de manière asynchrone  
function gestionLocalisation(position) {  
    console.log("Latitude : " + position.coords.latitude);  
    console.log("Longitude : " + position.coords.longitude);  
}
```

Requêtes HTTP asynchrones

Les requêtes HTTP sont la base du développement web moderne. Elles permettent de récupérer des données à partir d'un serveur et de les afficher dans une page web.

Les fonctions asynchrones sont utilisées pour effectuer des opérations qui prennent du temps, comme les appels réseau.

Depuis le début de la formation, vous avez envoyé des requêtes HTTP à (GET, POST) via un formulaire HTML ou via un lien hypertexte. La page devait être rechargée pour afficher les données renvoyées par le serveur.

Avec JavaScript, vous pouvez envoyer des requêtes HTTP sans recharger la page en utilisant l'API **Fetch**. L'API Fetch est une API JavaScript qui permet d'envoyer des requêtes HTTP asynchrones et de traiter les réponses.

- [Documentation Fetch API](#)

Syntaxe de base

Pour démarrer une requête Fetch, on utilise la fonction **fetch** en lui passant l'URL de la ressource à récupérer. La fonction **fetch** renvoie une promesse qui est résolue lorsque la requête est terminée. Nous verrons les promesses plus en détail dans une prochaine session.

L'URL fait une requête vers un serveur qui renvoie une réponse. Cela peut être vers un fichier/serveur PHP, un service web infonuagique, une API tierce, etc.

La réponse peut être au format JSON, XML, HTML, texte, etc.

```
// Exemple de requête Fetch
fetch("https://api.example.com/data")
  .then(function (reponse) {
    // Traiter la réponse de la requête sous forme de JSON
    return reponse.json();
  })
  .then(function (donnees) {
    // Effectuer des opérations sur les données renvoyées
    console.log(donnees);
  });
```

Étapes pour effectuer une requête Fetch :

1. On écrit la fonction **fetch** en indiquant l'URL de la ressource à récupérer. La demande est envoyée au serveur et une promesse est renvoyée.
2. Ensuite, il faut chaîner une première méthode **then** pour traiter la réponse de la requête. La méthode **then** prend une fonction de rappel qui traite la réponse de la requête. Dans l'exemple ci-dessus, on utilise la méthode **json()** pour extraire les données de la réponse. On pourrait également utiliser les méthodes **text()**, **blob()**, **arrayBuffer()**, **formData()** pour traiter la réponse renvoyée par le serveur. **N'oubliez pas de retourner la réponse de la requête pour pouvoir l'utiliser dans la prochaine méthode **then**.**
3. Ensuite, on chaîne un deuxième **then** pour effectuer des opérations sur les données renvoyées. Dans l'exemple ci-dessus, on utilise la méthode **console.log** pour afficher les données renvoyées.
4. Enfin, on utilise la méthode **catch** pour gérer les erreurs qui peuvent survenir lors de la requête. Dans l'exemple ci-dessus, on utilise la méthode **console.error** pour afficher les erreurs.

Par défaut, la fonction **fetch** envoie une requête **GET**. Pour envoyer une requête **POST**, **PUT**, **DELETE**, on peut passer un objet de configuration en deuxième paramètre qui contient des données.

```
function init() {  
  //Exemple de requête Fetch  
  //On indique la route à laquelle on veut envoyer la requête  
  fetch("https://api.example.com/data")  
    .then(function (response) {  
      //Traiter la réponse de la requête sous forme de JSON  
      return response.json();  
    })  
    .then(function (data) {  
      //Effectuer des opérations sur les données renvoyées  
      //Appelez une fonction pour afficher les données renvoyées  
      afficherDonnees(data);  
    })  
    .catch(function (error) {  
      //Gérer les erreurs  
      console.error(error);  
    });  
}  
  
function afficherDonnees(data) {  
  //...Logique pour afficher les données renvoyées  
}
```

Ressources

[ColorCode - Requêtes asynchrones](#) [ColorCode - Promesses](#) [ColorCode - API Fetch](#)

[Promesses - Fireship](#)

[MDN - Fetch API](#) [MDN - Promesses](#) [MDN - Intro aux requêtes asynchrones](#)